

Taskit

A gentle, AI-guided decluttering coach for people who feel overwhelmed by their space, including people living with hoarding disorder.

Form factor	Phone-first Progressive Web App (installable, camera-driven)
Input	One photo of one area of a room
Output	A 10-15 minute guided session of tiny, highlighted steps with a supportive chat coach
Stack	Vanilla JS PWA + Express proxy; FastAPI backend; Claude (Opus / Sonnet / Haiku), optional Gemini; Grounding DINO + SAM 2.1 for pixel-accurate outlines
Privacy	Photos are processed in memory, never written to disk, and cleared when the session ends or after an hour idle

1. The Problem

Clutter is rarely a knowledge problem. People usually know *what* should happen; what stops them is the size of the picture. Looking at a whole room triggers overwhelm, and every item raises a decision (keep it? donate it? is it worth something?) that can freeze a person in place. For people with hoarding disorder or chronic disorganization this is compounded by shame, attachment to belongings, and real safety risks: blocked exits, food waste, items near heat sources, and unstable stacks.

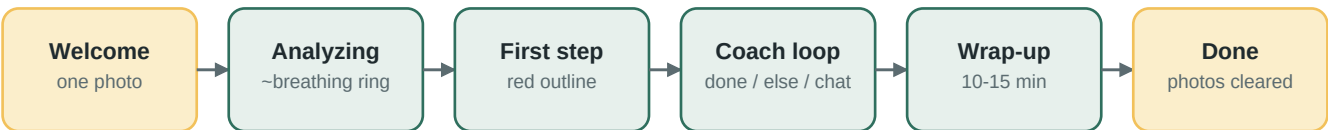
Existing tools make this worse in predictable ways:

- **Generic checklists** ("clean the kitchen") are too big to start and say nothing about *this* room.
- **Keep / donate / toss methods** demand constant decisions, and donate piles tend to sit and become new hazards.
- **Judgmental language** ("mess", "junk", "hoard") increases shame and makes people disengage.
- **Professional help** is valuable but expensive, scarce, and a big first step to take.

Taskit's premise: turn one photo into the single smallest, safest next action, show exactly which items it means, and keep the person company while they do it. No decisions about belongings, no shame, and permission to stop at any time.

2. User Cycle and Experience

The app is designed around a short, bounded session that ends in a feeling of success, whether the person completes three steps or ten.



Stage	What the person sees and does
Welcome	"One small step, then another." Three promises up front: about 15 minutes; stop whenever you like (stopping counts); your photo is private and never saved. They take a photo or choose one from their library.
Analyzing	A calm breathing-ring animation over their photo with rotating status lines. Cancel is always available and leaves nothing behind on the server.
Opening message	A warm welcome, a gear note only if something visible calls for it (e.g. gloves for food residue), an optional low-pressure offer of professional resources, and the first micro-task. The target items are outlined in red on their own photo.
Coach loop	A task card plus chat. Quick replies cover the common paths: Done , Something else (try a different step, this feels like too much, where do these go?), and support options (keep going gently, I need a minute, finish for today). Free text works too.
Closer photo	If a wide shot is too complex, the first step is "take a closer photo of the highlighted area"; the new photo continues the same session.
Wrap-up	As the time budget fills, the coach celebrates and offers to stop, mentioning the next step only if they have energy. When a photo's plan runs out, they can photograph another area.
Done	"You did something good today." A list of what they did, and confirmation that today's photos have been cleared.

Experience principles

- **Homes, not decisions.** Usable belongings are never judged; they just go to their home, like with like (cables to one labeled box, clothes to the laundry basket, dishes to the kitchen). Homes adapt to the room shown and to the furniture visible, and a home started earlier is referenced again later.
- **One micro-cluster at a time.** 1-4 nearby item groups, 1-5 minutes. The coach never lists the whole plan or says how much is left.
- **Kind but firm on safety.** Hazard steps (food waste, broken glass, blocked paths) cannot be skipped; "Something else" becomes a smaller version of the same step, explained gently.
- **Non-shaming language, enforced.** Prompts forbid words like mess, junk, or hoard, and the coach never reveals the internal clutter-scale level, even if asked.
- **Care comes first.** Messages that suggest self-harm immediately add crisis resources (988, findahelpline.com) and pause the session: "The cleaning can wait; you matter more."

3. Agent Workflow

Taskit uses a small team of specialized agents, each a single structured-output model call with its own prompt in backend/prompts/. **Code, not a model, runs the flow:** the orchestrator decides what runs when, validates every plan, and applies each intent through a deterministic session state machine. This keeps the agents focused and makes the system safe to put in front of vulnerable users.

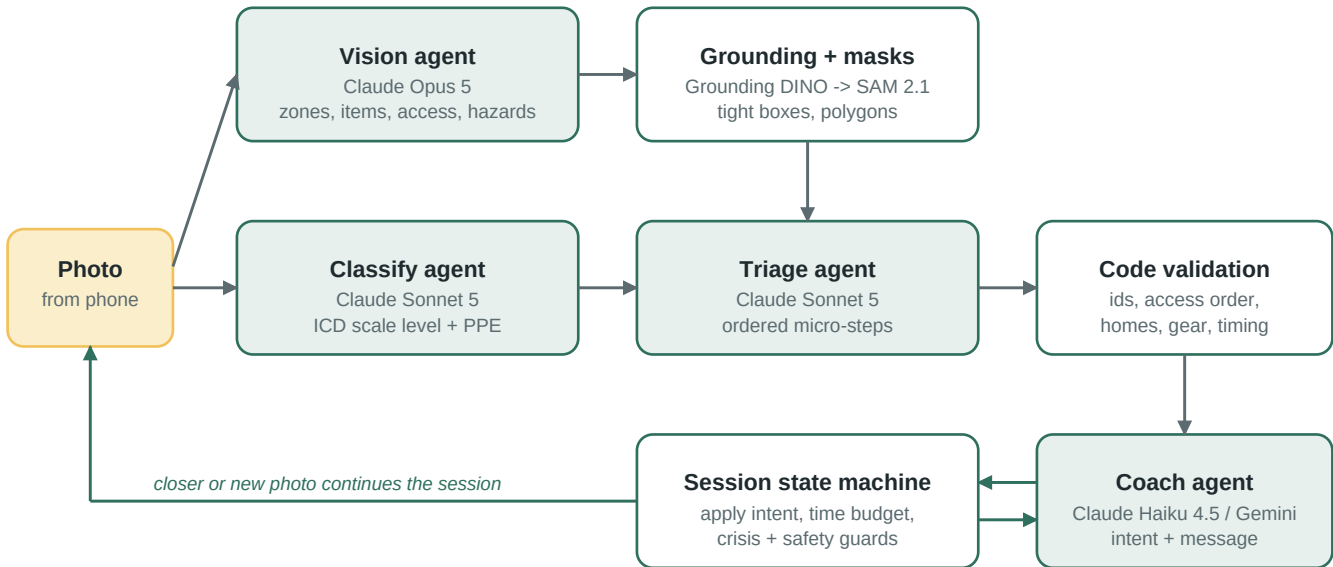


Figure: one photo fans out to Vision and Classify in parallel; their outputs feed Triage, whose plan is validated in code; the Coach then handles every turn until the session ends.

Agent	Model	Responsibility
Vision	Claude Opus 5	Reads the photo: room type and furniture, 3-8 zones, item groups with triage categories (trash, recycling, usable, sentimental, unsure), per-zone accessibility (<i>accessible</i> , <i>blocked_by</i> , <i>blocks_path</i>), visible hazards, a complexity score, and regions to exclude (people, pets, photos, screens).
Classify	Claude Sonnet 5	Rates the space on the ICD Clutter-Hoarding Scale (levels 1-5) from an embedded criteria knowledge base, using an escalation rule and listing what one photo cannot show. Runs in parallel with Vision; its failure is not fatal.
Triage	Claude Sonnet 5	Plans an ordered list of micro-steps (bag, recycle, group, set_aside, take_closer_photo) for ~12 minutes of work, plus extras for skips. Ordering: accessibility, then hazards, then blocked paths, then easy wins.
Coach	Claude Haiku 4.5 (or Gemini)	Every turn: classifies the reply as done / skip / smaller / support / pause / question / crisis and writes the next short, empathetic message for the current step.

How the pieces fit together

- **Start session.** Vision and Classify run concurrently in a thread pool. Vision's Claude pass is then refined locally: Grounding DINO finds tight boxes for each item phrase inside its zone, and SAM 2.1 turns them into masks and polygons for precise red outlines.
- **Plan and validate.** Triage proposes steps; code drops unknown ids, keeps sentimental and unsure items out of decision steps, never mixes usable belongings with trash, builds a dependency graph so no step lands in a zone before the zones blocking it are cleared, and re-estimates minutes from item counts.

- **Enrich.** Each usable item is matched to a home from `homes.json` (home, fallback, box label, practical tip, per-room overrides). Protective gear is reduced from the scale's full PPE list to a practical one: gloves only when visible hazards call for them, masks and more only at higher levels.
- **Each turn.** The coach sees a compact JSON context (current step, its homes, next step if done, alternative if skipped, smaller version, elapsed time, wrap-up flag, homes already set up, recent chat). The session applies the returned intent and advances.

Guardrails layered around the models

Guardrail	How it works
Hazard hold	A skip on a hazard step is forced to <i>smaller</i> (or <i>support</i>); if the model still returns skip, fixed safety wording replaces its message.
Language filters	Post-processing drops any sentence that mentions a scale level, severity or color code, or tells the person to tie off the trash bag before the session is ending.
Crisis detection	A regex check on every message overrides the intent to <i>crisis</i> , appends support resources, and pauses the session.
Graceful fallback	If the coach model is unreachable, keyword intent detection and templated messages keep the session moving; Gemini falls back to Claude on errors.
Privacy	Images live only in an in-memory session store with a one-hour idle TTL; DELETE clears it immediately. Excluded regions are blurred in rendered images.
Cancellation	A cancel event is checked at each stage boundary, and a session is only stored once complete, so cancelling leaves no photo behind.

4. Technical Implementation

Architecture

Layer	Implementation
Client (PWA)	<code>public/</code> : a single <code>index.html</code> with welcome, analyzing, session and done screens plus a camera sheet. <code>app.js</code> turns each <code>SessionView</code> into coach bubbles, task cards, photo outlines and quick replies; <code>camera.js</code> uses <code>getUserMedia</code> with a native-picker fallback; a network-first service worker never caches coach calls.
Web server	<code>server.js</code> (Express) serves the PWA and proxies <code>/backend/*</code> to FastAPI.
API	FastAPI (<code>app/main.py</code>). Sync orchestration functions run in a threadpool, so the same code powers the API and the CLI tools.
Orchestration	<code>orchestrator.py</code> (flow, guards, fallbacks), <code>session.py</code> (state machine, time budget, TTL store), <code>triage.py</code> (planning + validation).
Vision pipeline	<code>pipeline.py</code> : Claude scene pass -> Grounding DINO (<code>grounding.py</code>) -> SAM 2.1 (<code>segment.py</code>) -> <code>AnalysisResult</code> . Filters oversized and off-hint detections, applies NMS, and picks a focus cluster while dropping outliers. A shared re-entrant GPU lock keeps concurrent requests from colliding on Apple MPS.
Rendering	<code>render.py</code> draws the step image: light blur outside the focus box, a red frame, and thinner outlines on each item; excluded regions are blurred.

Layer	Implementation
LLM layer	<code>llm.py</code> : one shared <code>claude_parse</code> call using Pydantic structured outputs, adaptive thinking where useful, server-side model fallback, and typed errors (unusable answer vs. unreachable).
Schemas	<code>schemas.py</code> : Pydantic models for zones, items, instances, triage steps, assessment, coach replies, and the <code>SessionView</code> the client renders.

API surface

Endpoint	Purpose
POST <code>/sessions</code>	Multipart photo in; returns the first <code>SessionView</code> (coach message, current step with polygons, focus box and image URL, progress, gear).
POST <code>/sessions/{id}/messages</code>	<code>{text, action}</code> : buttons send an action, free text goes to the coach.
POST <code>/sessions/{id}/photo</code>	A closer or new-area photo continues the same session.
GET <code>.../steps/{step}/focus.jpg</code>	The highlighted step image (outline or box style).
DELETE <code>/sessions/{id}</code>	Forget the session and its photos immediately.
POST <code>/analyze, /render</code>	Lower-level single-photo analysis and rendering.

Key design decisions

- **Models reason, code decides.** Every model output is structured and validated; ordering, safety holds and state transitions are deterministic and testable (`tests/test_core.py`, `tests/test_session.py`).
- **Right model per job.** The most capable model handles the one-shot visual reasoning; a mid-tier model plans and classifies; a fast, inexpensive model handles low-latency chat turns. All are configurable via environment variables, and the coach can be swapped to Gemini for comparison.
- **Hybrid vision.** A VLM understands *what* is in the scene and why it matters; open-vocabulary detection and segmentation make the outlines precise enough to point at specific objects on a phone screen. A `--vlm-only` mode runs without local models.
- **Domain knowledge as data.** The ICD scale criteria and the homes catalog live in editable prompt and JSON files rather than code, so the behavior can be tuned without redeploying logic.
- **Developer tooling.** `analyze_cli.py` and `session_cli.py` run the full pipeline and an interactive session from the terminal, with cached JSON replay to skip API calls.

In one sentence: Taskit combines multi-agent AI with deterministic safety logic to turn an overwhelming room into one small, highlighted, judgment-free task at a time.